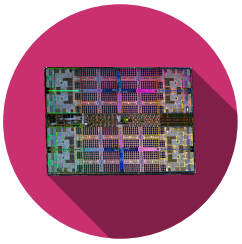


Single-Cycle RISC-V

Table des matières

1 Introduction 1

2 Architecture 2
2.1 Analyse 2

3 Control Unit 2
3.1 Réalisation 3

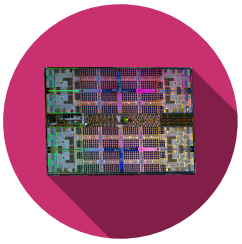
4 Simulation 3
4.1 Analyse 3

5 Déploiement (Facultatif) 3

Acronymes 4

1 Introduction

Ce laboratoire permet d'approcher l'architecture RISC-V à l'aide d'un processeur supportant un set d'instructions réduit, capable d'exécuter un petit programme assembleur en ne nécessitant qu'un seul coup de clock par instruction (single-cycle). Nommé `HEIRV32_SC`, ce dernier peut être simulé ou déployé directement sur une puce FPGA. La connexion avec le monde extérieur peut se faire à l'aide de boutons et de leds.



2 Architecture

Hormis l'incrémentation du program counter, le reste du circuit fonctionne de manière purement combinatoire. Le circuit est le suivant :

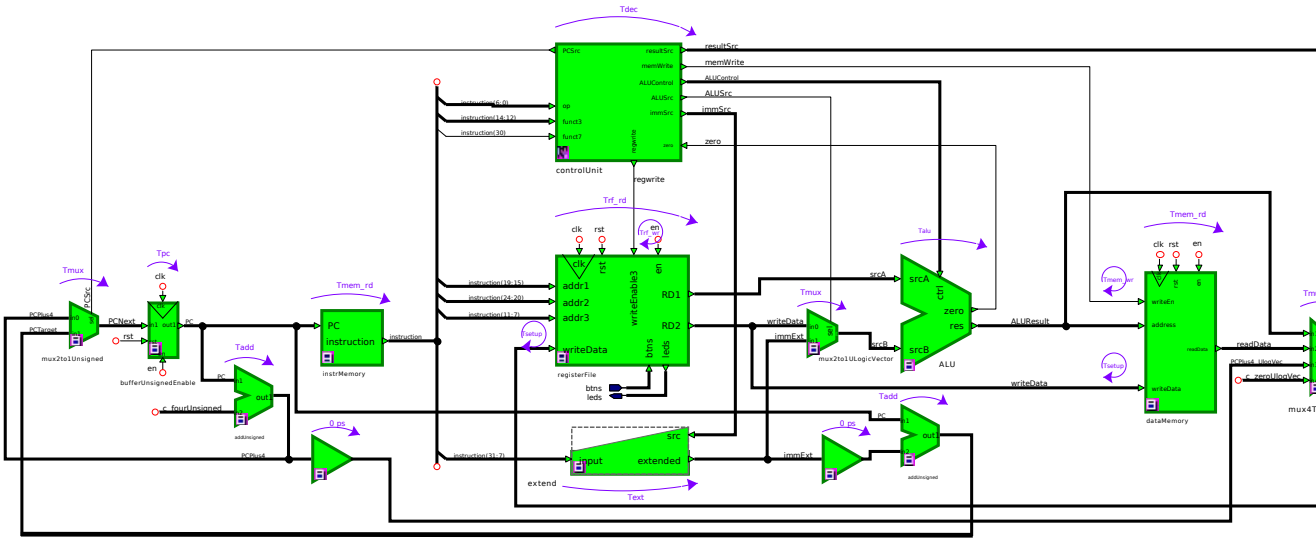


FIGURE 1 – Top level

2.1 Analyse

Déterminez les diverses fonctionnalités du circuit :

1. Quel est le rôle de chaque bloc ?
2. Identifiez les 5 étapes du pipeline RISC-V et quels blocs y sont liés.
3. A quoi servent chaque signaux ? Quelle est leur taille ?
4. Comment sont configurés les signaux du bloc **controlUnit** pour les instructions **or, addi, sw, beq, jal** ?

3 Control Unit

Une des principale caractéristique différençiant un type de processeur d'un autre est la façon dont sont encodées les instructions, liée à la structure interne. Par exemple, l'opération d'addition d'un nombre avec un registre existe tout autant dans l'architecture RISC-V qu'en **x86** ou **ARM**, mais l'opcode généré est tout autre. Pour l'opération **addi x0 x0 1** en RISC-V (**add r0, r0, 1** en ARM), le résultat est :

- ARM : 0x 01 00 80 E2
- RISC-V : 0x 00 10 00 13

Le bloc **controlUnit** est celui chargé de configurer le circuit selon l'instruction donnée.

3.1 Réalisation

Complétez le bloc **controlUnit** afin de supporter les instructions suivantes :

- R-type : add, sub, slt, or, and
- I-type : addi, slti, ori, andi, lw
- S-type : sw
- B-type : beq
- J-type : jal

4 Simulation

Sous le dossier **Simulation** se trouvent les codes utilisés pour simuler et déployer le système, en trois formats :

- .s : code assembleur, en clair
- .bin : code compilé ; peut être analysé à l'aide d'outils comme Ghidra ou de désassembleurs en ligne
- .txt : version compilée compatible avec la mémoire de la FPGA pour le déploiement

Le code est compilé à l'aide du programme **HEIRV32-ASM**, disponible avec sa documentation sous le dossier du même nom.

Le testeur **heirv32_sc_tb** utilise le code **code_sim.bin** lors de la simulation.

4.1 Analyse

Analysez dans un premier temps le code **code_sim.s** afin d'en comprendre le contenu. Est-ce un bon candidat pour tester votre système ? Effectuez les modifications nécessaires si besoin. Déterminez le temps nécessaire pour effectuer la boucle du programme une fois, sachant que le système est cadencé à **66 MHz**.

Lancez la simulation et confirmez le fonctionnement du système.

Répondez aux questions suivantes :

1. Quelles sont les faiblesses de cette architecture single-cycle ?
2. Quelle est la vitesse de clock maximale envisageable ?
3. Déterminez l'instruction la plus longue et retracez son chemin.

> Les temps de chaque sous-parties du système sont donnés dans le top-level.

5 Déploiement (Facultatif)

Lors du déploiement sur la FPGA, le système sélectionne le code **code_sc.*** comme source. Analysez et comprenez le code en question. Que devrait-il arriver ?

La librairie **Board** contient le bloc nécessaire au déploiement. Ouvrez-le et analysez le circuit : comment fonctionnent **en_step_n** et **en_full_n** ?

Le fichier **Board/concat/ebs2_heirv32.ucf** contient la définition et emplacements des pins du système. Le branchement de l'électronique se fait selon l'image suivante :

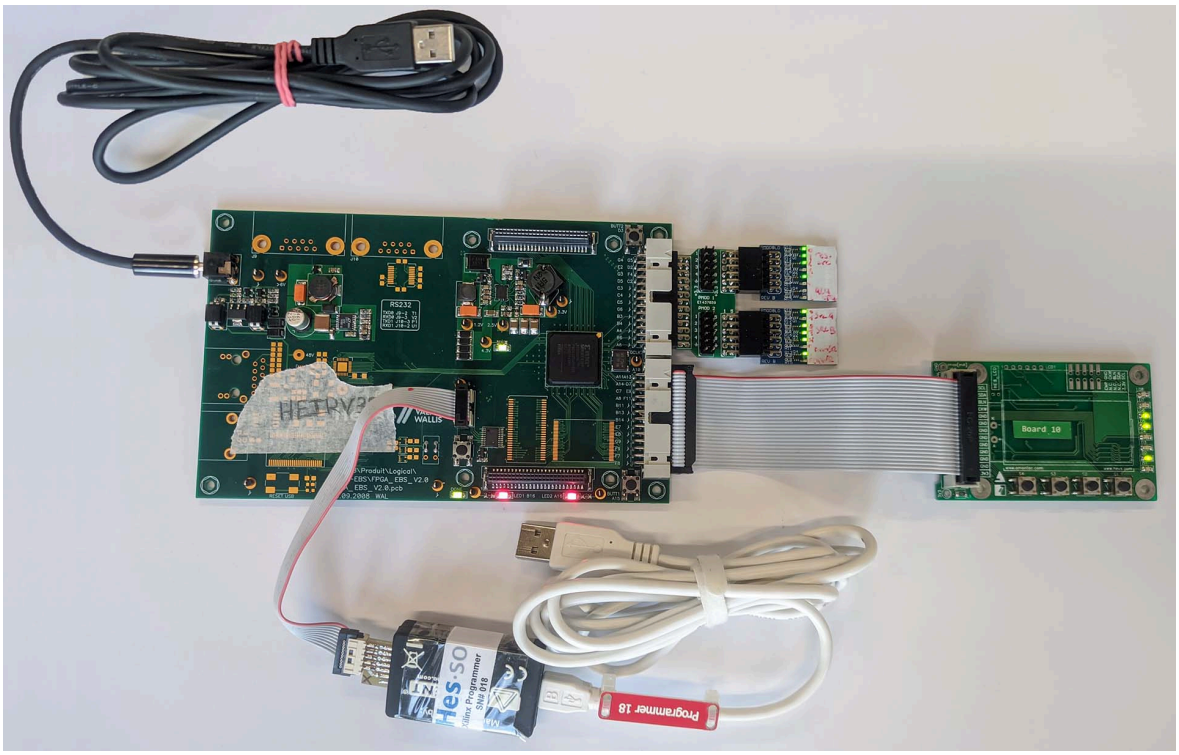


FIGURE 2 – Branchement du matériel

Déployez et testez le système sur la FPGA.

Au démarrage du système, l'entrée **en** de HEIRV32_SC est à 0 !

Acronymes

FPGA **F**ield **P**rogrammable **G**ates **A**rray. 1, 3, 4

RISC-V **R**educed **i**nstruction **s**et **c**omputer **V**. 1, 2